Water Jug Problems

1

```
from collections import deque

def water_jug():
    jug1, jug2 = 4, 3
    target = 2
    queue = deque([(0, 0, [])])
    visited = set()

    while queue:
        a, b, path = queue.popleft()

        if (a, b) in visited:
            continue
        visited.add((a, b))
        path = path + [(a, b)]

        if a == target or b == target:
            print("Steps:")
            for step in path:
                print(step)
            return

        states = [
            (jug1, b), (a, jug2), (0, b), (a, 0),
            (a - min(a, jug2 - b), b + min(a, jug2 - b)),
            (a + min(b, jug1 - a), b - min(b, jug1 - a))
```

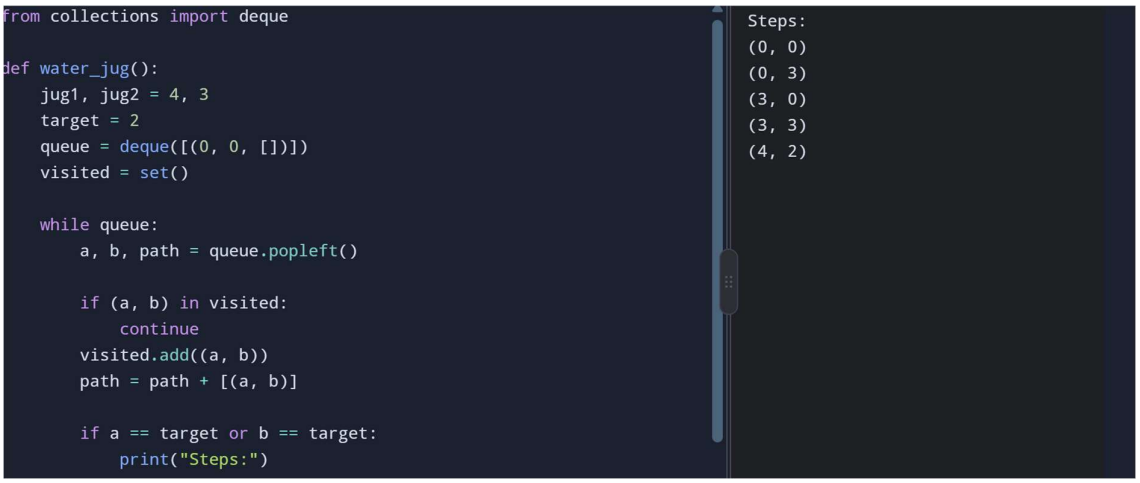
]

for state in states:

if state not in visited:

queue.append((state[0], state[1], path))

water_jug()



```
from collections import deque

def water_jug():
    jug1, jug2 = 4, 3
    target = 2
    queue = deque([(0, 0, [])])
    visited = set()

    while queue:
        a, b, path = queue.popleft()

        if (a, b) in visited:
            continue
        visited.add((a, b))
        path = path + [(a, b)]

        if a == target or b == target:
            print("Steps:")
```

```
Steps:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

2

from collections import deque

def water_jug():

cap1, cap2 = 5, 3

target = 4

queue = deque([(0, 0)])

visited = set()

while queue:

a, b = queue.popleft()

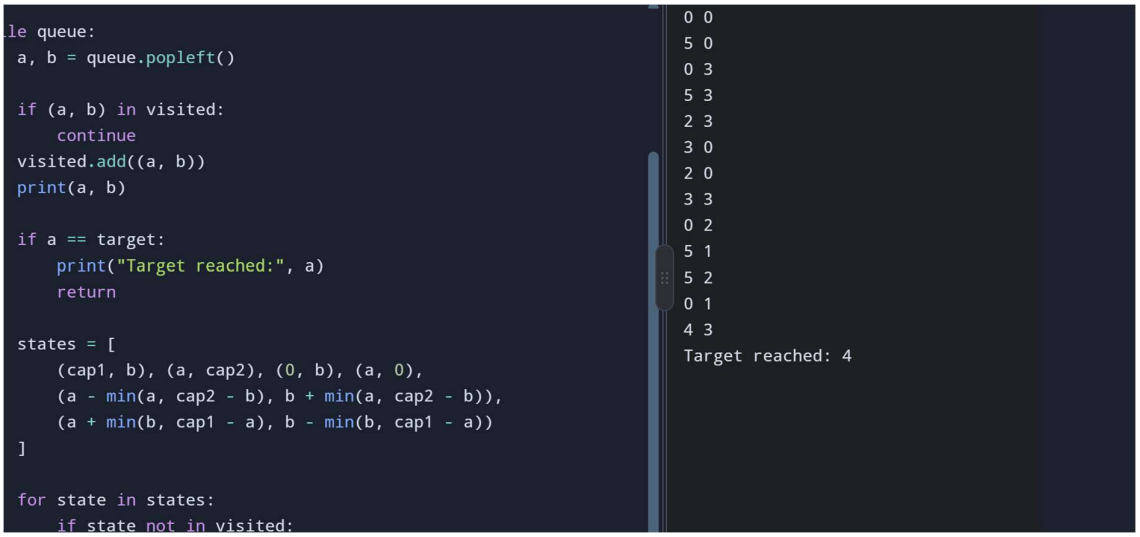
```
        if (a, b) in visited:
            continue
        visited.add((a, b))
        print(a, b)

        if a == target:
            print("Target reached:", a)
            return

        states = [
            (cap1, b), (a, cap2), (0, b), (a, 0),
            (a - min(a, cap2 - b), b + min(a, cap2 - b)),
            (a + min(b, cap1 - a), b - min(b, cap1 - a))
        ]

        for state in states:
            if state not in visited:
                queue.append(state)

water_jug()
```



3

from collections import deque

```
def water_jug():
    cap1, cap2 = 3, 5
    target = 1
    queue = deque([(0, 0)])
    visited = set()

    while queue:
        a, b = queue.popleft()

        if (a, b) in visited:
            continue
        visited.add((a, b))

        print("Jug states:", a, b)

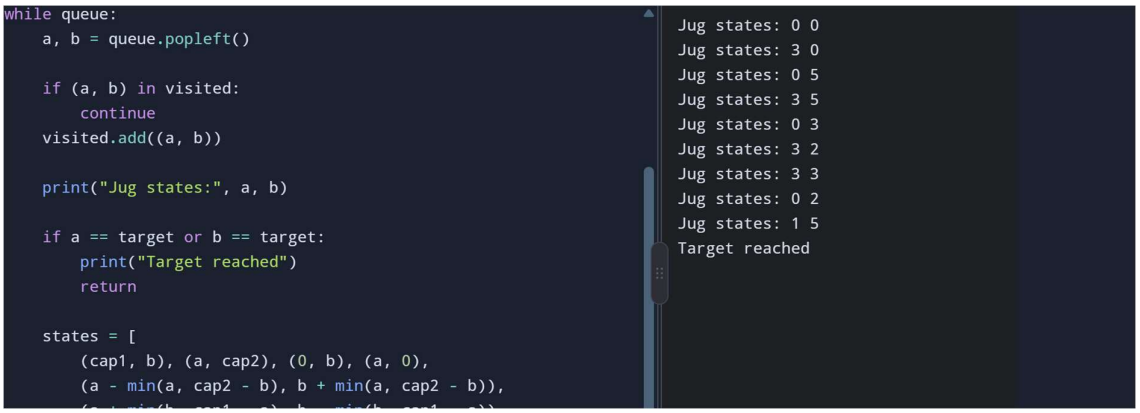
        if a == target or b == target:
```

```
        print("Target reached")
        return

    states = [
        (cap1, b), (a, cap2), (0, b), (a, 0),
        (a - min(a, cap2 - b), b + min(a, cap2 - b)),
        (a + min(b, cap1 - a), b - min(b, cap1 - a))
    ]

    for state in states:
        if state not in visited:
            queue.append(state)
```

water_jug()



4

```
from collections import deque


def water_jug():
    cap1, cap2 = 7, 4
    target = 5
```

```
    queue = deque([(0, 0)])
    visited = set()

    while queue:
        a, b = queue.popleft()

        if (a, b) in visited:
            continue
        visited.add((a, b))

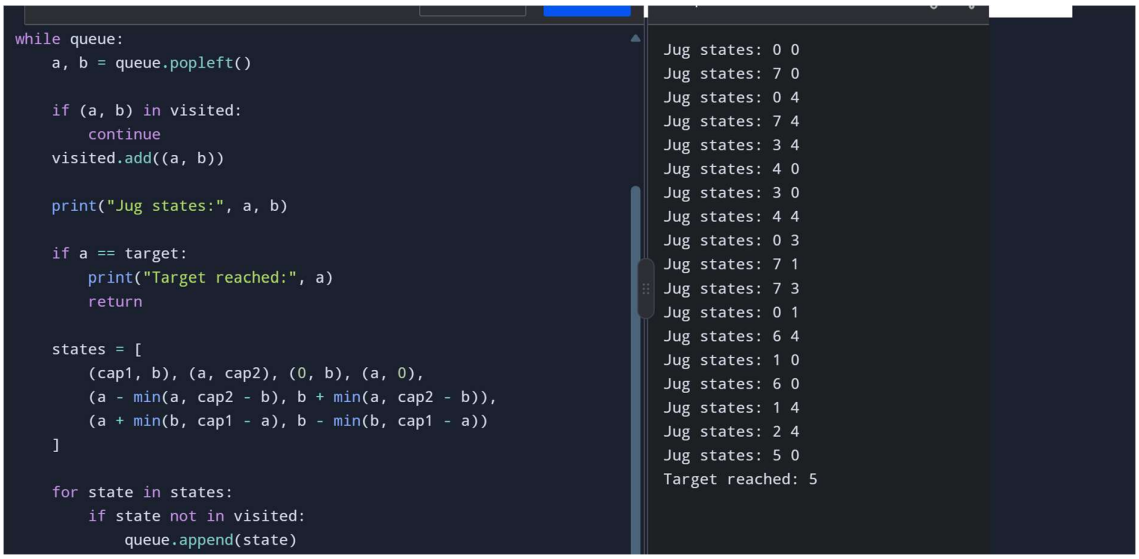
        print("Jug states:", a, b)

        if a == target:
            print("Target reached:", a)
            return

        states = [
            (cap1, b), (a, cap2), (0, b), (a, 0),
            (a - min(a, cap2 - b), b + min(a, cap2 - b)),
            (a + min(b, cap1 - a), b - min(b, cap1 - a))
        ]

        for state in states:
            if state not in visited:
                queue.append(state)

water_jug()
```



```
while queue:
    a, b = queue.popleft()

    if (a, b) in visited:
        continue
    visited.add((a, b))

    print("Jug states:", a, b)

    if a == target:
        print("Target reached:", a)
        return

    states = [
        (cap1, b), (a, cap2), (0, b), (a, 0),
        (a - min(a, cap2 - b), b + min(a, cap2 - b)),
        (a + min(b, cap1 - a), b - min(b, cap1 - a))
    ]

    for state in states:
        if state not in visited:
            queue.append(state)
```

```
Jug states: 0 0
Jug states: 7 0
Jug states: 0 4
Jug states: 7 4
Jug states: 3 4
Jug states: 4 0
Jug states: 3 0
Jug states: 4 4
Jug states: 0 3
Jug states: 7 1
Jug states: 7 3
Jug states: 0 1
Jug states: 6 4
Jug states: 1 0
Jug states: 6 0
Jug states: 1 4
Jug states: 2 4
Jug states: 5 0
Target reached: 5
```